

FETCH → AXIOS MIGRATION

Every call site, explained — companion to AXIOS-MIGRATION-EXPLAINED.md

1. GET lists/detail → 2. POST create → 3. PATCH update → 4. PUT replace → 5. DELETE remove → 6. Auth calls →
7. Interceptor safety net

1 Why this migration happened

Three disagreements between fetch and axios

CONCERN	FETCH	API (AXIOS)
Failure signal	Resolves normally even on 4xx/5xx — you must check <code>res.ok</code> yourself	Rejects the promise on any non-2xx status, so a plain <code>try/catch</code> catches it
Body parsing	Manual <code>await res.json()</code>	Already-parsed JSON on <code>response.data</code>
Expired access token (401)	Caller sees the failure directly	Interceptor may silently refresh & retry first — caller only sees failure if that <i>also</i> fails

2 The shape of the change, everywhere

Before

```
const res = await fetch("/api/consumers");
if (!res.ok) throw new Error("Request failed");
const data = await res.json();
```

After

```
const { data } = await api.get("/consumers");
// baseUrl "/api" is already set once,
// in lib/axios.ts
```

3 Where every call actually goes



Full detail on the interceptor itself — de-duplicated refresh, retry-once, hard redirect on failure — lives in AXIOS-INTERCEPTORS-EXPLAINED.md. This document covers everywhere that engine gets *used*.

1 useConsumers.ts — list + query params

client/src/features/consumers/get/useConsumers.ts

```
const { data } = await api.get<ConsumersResponse>("/consumers", {
  params: { search: search || undefined, page, pageSize },
});
setConsumers(data.consumers);
setTotal(data.total);
```

- `api.get<ConsumersResponse>` — the generic types data, so `data.consumers/data.total` are checked instead of any.
- `params: { ... }` — axios builds `?search=...&page=...&pageSize=...` itself.
- `search || undefined` is the key detail: an empty search box should mean "no filter." Axios *drops* any key whose value is undefined. A raw `URLSearchParams` would instead send the literal text "undefined" as the filter value.
- The `try / catch / finally` + ignore race-guard scaffolding is unchanged — axios only replaces the network-call line, not the surrounding state machine.

2 useConsumerDetail.ts — path param instead of query

client/src/features/consumers/get/useConsumerDetail.ts

```
const { data } = await api.get<ConsumerResponse>(`/consumers/${id}`);
```

- No params option — the id is already baked into the URL path, not a filter on it.
- Same `useEffect` + ignore flag pattern as the list hook: re-runs when `id` changes, and a stale in-flight response can't overwrite a newer one.

→ GET request/response, end to end

SUCCESS PATH

Component → `api.get("/consumers", {params})` → GET `/api/consumers?search=...` → 200 + JSON → `setConsumers(data)`

ACCESS TOKEN EXPIRED MID-REQUEST

`api.get(...)` → 401 → interceptor: refresh + retry → 200 (transparent)

3 useCreateConsumer.ts — POST

WIRED

client/src/features/consumers/post/useCreateConsumer.ts

```
const { data } = await api.post<ConsumerResponse>("/consumers", input);
return data.consumer;
```

- The body is axios's **second positional argument** — not a params-style options object, and not a manually JSON.stringify'd fetch body.
- Axios serializes input and sets Content-Type: application/json itself — both were repeated boilerplate in the old fetch version.

4 useUpdateConsumer.ts — PATCH

STUBBED

.../patch/useUpdateConsumer.ts

```
/*
const { data } = await
api.patch<ConsumerResponse>(
  `/consumers/${id}`, input); // ONLY changed
fields
return data.consumer;
*/
throw new Error("PATCH not wired — DEMO STEP 4");
```

- Partial update — send only the fields that changed.

5 useReplaceConsumer.ts — PUT

STUBBED

.../put/useReplaceConsumer.ts

```
/*
const { data } = await api.put<ConsumerResponse>(
  `/consumers/${id}`, input); // the WHOLE object
return data.consumer;
*/
throw new Error("PUT not wired — DEMO STEP 3");
```

- Full replace — every field, every time.

6 useDeleteConsumer.ts — DELETE

STUBBED

client/src/features/consumers/delete/useDeleteConsumer.ts

```
/* await api.delete(`/consumers/${id}`); */
throw new Error("DELETE not wired — DEMO STEP 5");
```

- No body argument at all — DELETE identifies what to remove entirely through the URL.
- Enabling any stub later is a two-line change: delete the throw, uncomment the real call above it.

HOOK	METHOD	BODY ARG?	STATUS
useConsumers.ts	GET	—	wired
useConsumerDetail.ts	GET	—	wired
useCreateConsumer.ts	POST	full object	wired
useUpdateConsumer.ts	PATCH	changed fields only	stubbed
useReplaceConsumer.ts	PUT	full object	stubbed
useDeleteConsumer.ts	DELETE	none	stubbed

7 Where auth calls the shared instance

useAuth.tsx — session check

```
const { data } = await api.get<{user}>
("/auth/me");
setUser(data.user);
```

- Same-origin via the Vite proxy — the httpOnly access_token cookie is attached automatically, on fetch or on axios alike. Axios only changes *how failure is detected*.

login-page.tsx SKIP-LIST

```
await api.post("/auth/login", { email, password
});
// catch: isAxiosError(err) narrows
// err.response.data
```

- A 401 here means "wrong password," not "expired token" — the interceptor does **not** try to refresh it.

signup-page.tsx SKIP-LIST

```
await api.post("/auth/signup", { email, password
});
```

- Same isAxiosError narrowing as login. A duplicate-email failure isn't a token problem either.

dashboard-header.tsx — logout

```
await api.post("/auth/logout");
```

- No response body to read — success just means the cookies are now cleared server-side.

8 The interceptor's five-step story

1 · 2 — NORMAL REQUEST

Your app → access token cookie → Server → 200 + data → Your app

3 — TOKEN GOES STALE

Your app → same request, old token → Server → 401 TOKEN_EXPIRED → interceptor fires

4 · 5 — SILENT RECOVERY

interceptor → POST /auth/refresh → Server → new cookie → retry original request → 200, caller none the wiser

If refresh itself fails (refresh token also expired/invalid) → interceptor gives up and hard-redirects to /login via window.location.href — plain module code with no access to useNavigate().

9 What changed vs. what to check

What migrating to axios buys you

- ✓ Rejected promises on 4xx/5xx — no forgotten `res.ok` check
- ✓ Parsed JSON on `response.data` automatically
- ✓ One shared interceptor recovers expired tokens transparently
- ✓ `isAxiosError` narrows error shape for real server messages
- ✓ Query params built via `params`, no manual string-building

Things to double-check on a new call site

- ✗ Forgetting `try/catch` → unhandled rejection instead of a quiet `if (!res.ok)` miss
- ✗ New authenticated endpoint that shouldn't auto-refresh on 401 → add it to `AUTH_ENDPOINTS_TO_SKIP`
- ✗ Passing the body as an options object instead of the second positional argument on `post/patch/put`
- ✗ Using raw `axios` instead of the shared `api` instance — skips `baseUrl` and the interceptor entirely

10 Full call-site map



See also: `AXIOS-MIGRATION-EXPLAINED.md` (full prose walkthrough), `AXIOS-INTERCEPTORS-EXPLAINED.md` (the interceptor engine itself), `PUT-PATCH-EXPLAINED.md`, and milestone **M11** in `AUTH-LEARNING-MILESTONES.md`.